NumPy Practice Questions

This notebook covers essential NumPy operations including array creation, manipulation, statistical operations, and matrix operations.

In [ ]:
```
import numpy as np
```

1. Create an Array and Extract Even Numbers

Create a NumPy array and extract all even numbers from it using boolean indexing.

In [2]:
```
# Create a 1D array with random integers
np.random.seed(42)
arr = np.random.randint(1, 50, size=20)
print("Original array:")
print(arr)
print()

# Extract even numbers using boolean indexing
even_mask = arr % 2 == 0
even_numbers = arr[even_mask]
print("Even numbers:")
print(even_numbers)
print()

# Extract odd numbers
odd_numbers = arr[arr % 2 != 0]
print("Odd numbers:")
print(odd_numbers)
print()

# Additional operations
print(f"Total numbers: {len(arr)}")
print(f"Even count: {len(even_numbers)}")
print(f"Odd count: {len(odd_numbers)}")
print(f"Sum of even numbers: {np.sum(even_numbers)}")
print(f"Sum of odd numbers: {np.sum(odd_numbers)}")
```

```
Original array:
[39 29 15 43  8 21 39 19 23 11 11 24 36 40 24  3 22  2 24 44]

Even numbers:
[ 8 24 36 40 24 22  2 24 44]

Odd numbers:
[39 29 15 43 21 39 19 23 11 11  3]

Total numbers: 20
Even count: 9
Odd count: 11
Sum of even numbers: 224
Sum of odd numbers: 253
```

2. Identity Matrix and Random Matrix

Create an identity matrix and multiply it with a random matrix to demonstrate matrix operations.

In [3]:
```python
# Create a 4x4 identity matrix
identity_matrix = np.eye(4)
print("4x4 Identity Matrix:")
print(identity_matrix)
print()

# Create a 4x4 random matrix
np.random.seed(42)
random_matrix = np.random.rand(4, 4) * 10  # Random values between 0-10
print("4x4 Random Matrix:")
print(random_matrix)
print()

# Matrix multiplication with identity (should return original matrix)
result_identity = np.dot(identity_matrix, random_matrix)
print("Identity × Random Matrix:")
print(result_identity)
print()

# Verify they are the same
print("Are they equal?", np.allclose(random_matrix, result_identity))
print()

# Additional matrix operations
print("Matrix determinant:", np.linalg.det(random_matrix))
print("Matrix trace:", np.trace(random_matrix))
print("Matrix transpose:")
print(random_matrix.T)
```

```
4x4 Identity Matrix:
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

4x4 Random Matrix:
[[3.74540119 9.50714306 7.31993942 5.98658484]
 [1.5601864  1.5599452  0.58083612 8.66176146]
 [6.01115012 7.08072578 0.20584494 9.69909852]
 [8.32442641 2.12339111 1.81824967 1.8340451 ]]

Identity × Random Matrix:
[[3.74540119 9.50714306 7.31993942 5.98658484]
 [1.5601864  1.5599452  0.58083612 8.66176146]
 [6.01115012 7.08072578 0.20584494 9.69909852]
 [8.32442641 2.12339111 1.81824967 1.8340451 ]]

Are they equal? True

Matrix determinant: -2656.9951655496525
Matrix trace: 7.345236433328013
Matrix transpose:
[[3.74540119 1.5601864  6.01115012 8.32442641]
 [9.50714306 1.5599452  7.08072578 2.12339111]
 [7.31993942 0.58083612 0.20584494 1.81824967]
 [5.98658484 8.66176146 9.69909852 1.8340451 ]]
```

3. Reshape and Flatten Arrays

Demonstrate array reshaping and flattening operations with different techniques.

In [4]:
```python
# Create a 1D array
original_array = np.arange(1, 25)  # Numbers 1 to 24
print("Original 1D array:")
print(original_array)
print(f"Shape: {original_array.shape}")
print()

# Reshape to different dimensions
# 2D array (4x6)
array_2d = original_array.reshape(4, 6)
print("Reshaped to 4x6:")
print(array_2d)
print(f"Shape: {array_2d.shape}")
print()

# 3D array (2x3x4)
array_3d = original_array.reshape(2, 3, 4)
print("Reshaped to 2x3x4:")
print(array_3d)
print(f"Shape: {array_3d.shape}")
print()
```

```python
# Flatten the 3D array back to 1D
flattened_array = array_3d.flatten()
print("Flattened array:")
print(flattened_array)
print(f"Shape: {flattened_array.shape}")
print()

# Using ravel (returns a view if possible)
raveled_array = array_3d.ravel()
print("Raveled array:")
print(raveled_array)
print()

# Reshape with -1 (automatic dimension calculation)
auto_reshaped = original_array.reshape(6, -1)
print("Auto-reshaped to 6x? (6x4):")
print(auto_reshaped)
print(f"Shape: {auto_reshaped.shape}")
```

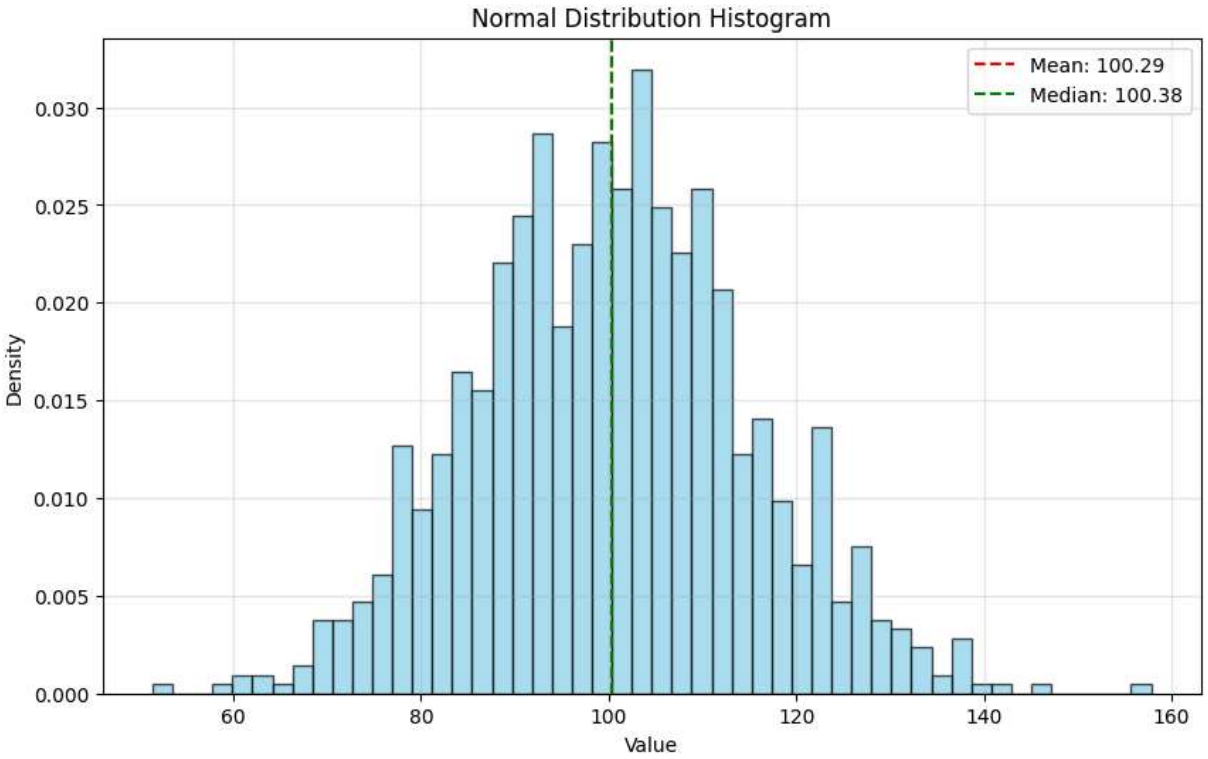
```
Original 1D array:
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]
Shape: (24,)

Reshaped to 4x6:
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]]
Shape: (4, 6)

Reshaped to 2x3x4:
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[13 14 15 16]
  [17 18 19 20]
  [21 22 23 24]]]
Shape: (2, 3, 4)

Flattened array:
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]
Shape: (24,)

Raveled array:
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]

Auto-reshaped to 6x? (6x4):
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]
 [21 22 23 24]]
Shape: (6, 4)
```

4. Normal Distribution and Statistics

Generate data from normal distribution and calculate various statistical measures.

In [5]:
```
# Generate normal distribution data
np.random.seed(42)
mean = 100
std_dev = 15
size = 1000

normal_data = np.random.normal(mean, std_dev, size)
print(f"Generated {size} samples from normal distribution")
print(f"Theoretical mean: {mean}, std: {std_dev}")
print()

# Calculate statistics
print("Statistical Measures:")
print(f"Sample mean: {np.mean(normal_data):.2f}")
print(f"Sample std: {np.std(normal_data):.2f}")
print(f"Sample variance: {np.var(normal_data):.2f}")
print(f"Median: {np.median(normal_data):.2f}")
print(f"Minimum: {np.min(normal_data):.2f}")
print(f"Maximum: {np.max(normal_data):.2f}")
print(f"Range: {np.ptp(normal_data):.2f}")
print()

# Percentiles
percentiles = [25, 50, 75, 90, 95, 99]
print("Percentiles:")
for p in percentiles:
    value = np.percentile(normal_data, p)
    print(f"{p}th percentile: {value:.2f}")
print()

# Plot histogram
plt.figure(figsize=(10, 6))
plt.hist(normal_data, bins=50, density=True, alpha=0.7, color='skyblue', edgecolor=
plt.axvline(np.mean(normal_data), color='red', linestyle='--', label=f'Mean: {np.me
plt.axvline(np.median(normal_data), color='green', linestyle='--', label=f'Median:
plt.xlabel('Value')
plt.ylabel('Density')
plt.title('Normal Distribution Histogram')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
Generated 1000 samples from normal distribution
Theoretical mean: 100, std: 15

Statistical Measures:
Sample mean: 100.29
Sample std: 14.68
Sample variance: 215.53
Median: 100.38
Minimum: 51.38
Maximum: 157.79
Range: 106.41

Percentiles:
25th percentile: 90.29
50th percentile: 100.38
75th percentile: 109.72
90th percentile: 119.58
95th percentile: 125.15
99th percentile: 134.74
```



5. Normalize Array

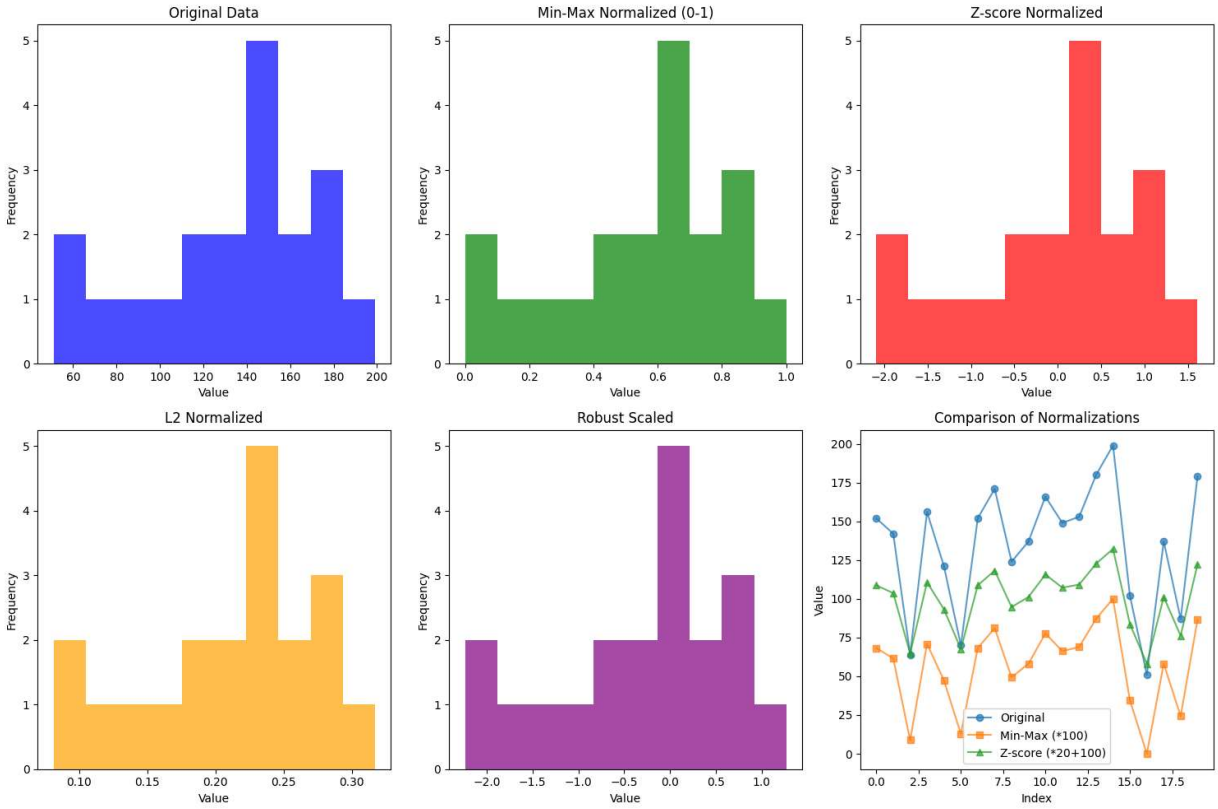
Demonstrate different normalization techniques for arrays including min-max scaling and z-score normalization.

In [6]:
```
# Create sample data with different scales
np.random.seed(42)
data = np.random.randint(50, 200, size=20).astype(float)
print("Original data:")
print(data)
```

```
print(f"Min: {np.min(data):.2f}, Max: {np.max(data):.2f}")
print(f"Mean: {np.mean(data):.2f}, Std: {np.std(data):.2f}")
print()

# 1. Min-Max Normalization (0 to 1)
min_max_normalized = (data - np.min(data)) / (np.max(data) - np.min(data))
print("Min-Max Normalized (0-1):")
print(min_max_normalized)
print(f"Min: {np.min(min_max_normalized):.2f}, Max: {np.max(min_max_normalized):.2f
print()

# 2. Min-Max Normalization to custom range (-1 to 1)
min_max_custom = 2 * (data - np.min(data)) / (np.max(data) - np.min(data)) - 1
print("Min-Max Normalized (-1 to 1):")
print(min_max_custom)
print(f"Min: {np.min(min_max_custom):.2f}, Max: {np.max(min_max_custom):.2f}")
print()

# 3. Z-score Normalization (Standardization)
z_score_normalized = (data - np.mean(data)) / np.std(data)
print("Z-score Normalized:")
print(z_score_normalized)
print(f"Mean: {np.mean(z_score_normalized):.2f}, Std: {np.std(z_score_normalized):.
print()

# 4. Unit Vector Normalization (L2 norm)
l2_normalized = data / np.linalg.norm(data)
print("L2 Normalized:")
print(l2_normalized)
print(f"L2 norm: {np.linalg.norm(l2_normalized):.2f}")
print()

# 5. Robust Scaling (using median and IQR)
median = np.median(data)
q75, q25 = np.percentile(data, [75, 25])
iqr = q75 - q25
robust_scaled = (data - median) / iqr
print("Robust Scaled:")
print(robust_scaled)
print(f"Median: {np.median(robust_scaled):.2f}")
print()

# Visualization
plt.figure(figsize=(15, 10))

# Original data
plt.subplot(2, 3, 1)
plt.hist(data, bins=10, alpha=0.7, color='blue')
plt.title('Original Data')
plt.xlabel('Value')
plt.ylabel('Frequency')

# Min-Max normalized
plt.subplot(2, 3, 2)
plt.hist(min_max_normalized, bins=10, alpha=0.7, color='green')
plt.title('Min-Max Normalized (0-1)')
```

```
plt.xlabel('Value')
plt.ylabel('Frequency')

# Z-score normalized
plt.subplot(2, 3, 3)
plt.hist(z_score_normalized, bins=10, alpha=0.7, color='red')
plt.title('Z-score Normalized')
plt.xlabel('Value')
plt.ylabel('Frequency')

# L2 normalized
plt.subplot(2, 3, 4)
plt.hist(l2_normalized, bins=10, alpha=0.7, color='orange')
plt.title('L2 Normalized')
plt.xlabel('Value')
plt.ylabel('Frequency')

# Robust scaled
plt.subplot(2, 3, 5)
plt.hist(robust_scaled, bins=10, alpha=0.7, color='purple')
plt.title('Robust Scaled')
plt.xlabel('Value')
plt.ylabel('Frequency')

# Comparison plot
plt.subplot(2, 3, 6)
x = np.arange(len(data))
plt.plot(x, data, 'o-', label='Original', alpha=0.7)
plt.plot(x, min_max_normalized * 100, 's-', label='Min-Max (*100)', alpha=0.7)
plt.plot(x, z_score_normalized * 20 + 100, '^-', label='Z-score (*20+100)', alpha=0
plt.title('Comparison of Normalizations')
plt.xlabel('Index')
plt.ylabel('Value')
plt.legend()

plt.tight_layout()
plt.show()
```

```
Original data:
[152. 142.  64. 156. 121.  70. 152. 171. 124. 137. 166. 149. 153. 180.
 199. 102.  51. 137.  87. 179.]
Min: 51.00, Max: 199.00
Mean: 134.60, Std: 39.96

Min-Max Normalized (0-1):
[0.68243243 0.61486486 0.08783784 0.70945946 0.47297297 0.12837838
 0.68243243 0.81081081 0.49324324 0.58108108 0.77702703 0.66216216
 0.68918919 0.87162162 1.         0.34459459 0.         0.58108108
 0.24324324 0.86486486]
Min: 0.00, Max: 1.00

Min-Max Normalized (-1 to 1):
[ 0.36486486  0.22972973 -0.82432432  0.41891892 -0.05405405 -0.74324324
  0.36486486  0.62162162 -0.01351351  0.16216216  0.55405405  0.32432432
  0.37837838  0.74324324  1.         -0.31081081 -1.          0.16216216
 -0.51351351  0.72972973]
Min: -1.00, Max: 1.00

Z-score Normalized:
[ 0.43541657  0.18517716 -1.76669021  0.53551233 -0.34032559 -1.61654656
  0.43541657  0.91087144 -0.26525377  0.06005746  0.78575173  0.36034474
  0.46044051  1.1360869   1.61154177 -0.81578046 -2.09200143  0.06005746
 -1.19113957  1.11106296]
Mean: 0.00, Std: 1.00

L2 Normalized:
[0.24206952 0.2261439  0.10192401 0.24843977 0.19270008 0.11147939
 0.24206952 0.27232821 0.19747777 0.21818108 0.2643654  0.23729183
 0.24366208 0.28666128 0.31691997 0.16244139 0.08122069 0.21818108
 0.13855295 0.28506871]
L2 norm: 1.00

Robust Scaled:
[ 0.15384615 -0.08284024 -1.92899408  0.24852071 -0.57988166 -1.78698225
  0.15384615  0.6035503  -0.50887574 -0.20118343  0.4852071   0.08284024
  0.17751479  0.81656805  1.26627219 -1.0295858  -2.23668639 -0.20118343
 -1.38461538  0.79289941]
Median: 0.00
```


Original Data
Min-Max Normalized (0-1)
Z-score Normalized
L2 Normalized
Robust Scaled
Comparison of Normalizations
Frequency
Value
Index
Original
Min-Max (*100)
Z-score (*20+100)